

Contents

Manuale d'uso — Pipeline Dataverse -> Archivematica	2
Indice	3
1. Panoramica del flusso	3
2. Prerequisiti	3
Software	3
Accesso a Dataverse	4
Accesso ad Archivematica	4
Permessi filesystem	4
Note su SSL	4
3. Configurazione — file .env	4
Contenuto completo del .env	4
Recuperare i valori mancanti	5
Priorita' delle variabili	5
Sicurezza	6
Errori comuni nel .env	6
4. Script 1 — scarica_dataverse.py	6
Cosa fa	6
Controllo dataset vuoti	7
Conservazione dell'originale (file tabellari)	7
Verifica di integrità dei file	7
Esportazione DCAT-AP e Schema.org	8
Opzioni da riga di comando	9
Esempi	10
Output a schermo	10
Comportamento idempotente	11
5. Script 2 — archivematica_ingest.py	11
Cosa fa	11
Gestione di timeout ed errori di rete transitori	12
Sorgente del processing config: copia locale versionata (RACCO- MANDATO)	12
Opzioni da riga di comando	13
Esempi	14
Output a schermo	15
6. Struttura del pacchetto generato	16
Formato metadata.csv (radice del pacchetto)	16
7. File di stato e report	16
riepilogo_download.json	16
doi_vuoti.json	17
stato_archivematica.json	17
Valori di status (stato_archivematica.json)	17
Riconciliazione dello stato — riconcilia_stato.py	18
Reset manuale di un pacchetto	18
Reset completo	18
8. Risoluzione dei problemi comuni	19

MissingSchema / URL non valido	19
302 Found sulla porta 8000	19
401 Unauthorized	19
404 Not Found sulla Transfer Source UUID	19
SSL Certificate Verify Failed	19
EXPORT_DCAT=true nel .env ma dcat.json non viene generato	19
[RISCARICO] ripetuti o “checksum non combacia” su un file . .	19
[FALLBACK] salvata derivata .tab	20
Ho ri-scaricato e mi ritrovo sia i .csv/.dta sia i vecchi .tab . . .	20
[ERRORE] Impossibile leggere il processing config	20
[ERRORE] Copia di processingMCP.xml nel pacchetto fallita (Permission denied sulla DESTINAZIONE)	21
Transfer REJECTED al riavvio	22
Read timed out durante transfer/ingest	22
Transfer “zombie” in Dashboard (in attesa, non rimovibili) . . .	22
Permission denied sulla copia di processingMCP.xml nei pacchetti (cartelle di proprieta’ altrui)	23
400 “Unable to determine the status of the unit”	23
Auto-approve non funziona	23
Elasticsearch non raggiungibile	23
9. Esempio di esecuzione completa	24
Configurazione iniziale (una tantum)	24
Esecuzione standard	24

Manuale d’uso — Pipeline Dataverse -> Archivematica

Questo manuale documenta l’uso di due script Python che automatizzano il trasferimento di dataset da **Dataverse UNIMI** (dataverse.unimi.it) ad **Archivematica** per la conservazione a lungo termine.

Script	Funzione
<code>scarica_dataverse.py</code>	Scarica dataset e metadati da Dataverse e li organizza in pacchetti compatibili con Archivematica
<code>archivematica_ingest.py</code>	Trasferisce ed esegue l’ingest dei pacchetti in Archivematica, con approvazione automatica del transfer
<code>riconcilia_stato.py</code>	Riconcilia i pacchetti “failed” del file di stato con gli AIP realmente presenti nello Storage Service (es. dopo approvazioni manuali)

Indice

1. Panoramica del flusso
 2. Prerequisiti
 3. Configurazione — file .env
 4. Script 1 — scarica_dataverse.py
 5. Script 2 — archivematica_ingest.py
 6. Struttura del pacchetto generato
 7. File di stato e report
 8. Risoluzione dei problemi comuni
 9. Esempio di esecuzione completa
-

1. Panoramica del flusso

```
dois.txt          scarica_dataverse.py          Transfer Source
(lista DOI)  -----> - controlla se vuoto  -----> Location
                                     (Archivematica)
                                     |
                                     v
                                     - scarica schema.json (opt.) archivematica_ingest.py
                                     -
verifica processing config (fail-fast)
                                     - copia processingMCP.xml
                                     - avvia transfer
                                     - auto-approva transfer
                                     - attende ingest
                                     - salva AIP UUID
```

Il flusso tipico e':

1. Si prepara `dois.txt` con l'elenco dei DOI da archiviare.
 2. Si esegue `scarica_dataverse.py`: scarica tutte le versioni di ciascun dataset, controlla che non siano vuoti, e li organizza nella Transfer Source Location di Archivematica.
 3. Si esegue `archivematica_ingest.py`: avvia il transfer e l'ingest per ogni pacchetto non ancora processato, in modo completamente automatico.
 4. Lo stato viene salvato in file JSON per le esecuzioni successive.
-

2. Prerequisiti

Software

```
pip install requests --break-system-packages
```

Accesso a Dataverse

- Una **API key Dataverse** (necessaria solo per dataset ad accesso ristretto; per dataset pubblici puo' essere lasciata vuota).

Accesso ad Archivematica

- **Dashboard API key** dell'utente Archivematica (es. `zfed`)
- **Storage Service API key** dell'utente Storage Service (es. `archivematica`)
- **UUID della Transfer Source Location**
- Accesso in lettura/scrittura alla Transfer Source Location sul filesystem

Permessi filesystem

```
sudo usermod -aG archivematica <utente>  
# Richiede logout/login per avere effetto
```

Note su SSL

Se Archivematica usa un certificato self-signed, impostare `SSL_VERIFY=false` nel `.env` o usare `--no-verify-ssl` da riga di comando.

URL tipici per le API: - Dashboard: `https://<host>` (porta 443) - Storage Service: `https://<host>:8000`

3. Configurazione — file `.env`

Tutti i parametri vanno nel file `.env` nella stessa cartella degli script. Il file viene caricato automaticamente all'avvio senza dipendenze esterne.

```
cp .env.template .env  
nano .env
```

Contenuto completo del `.env`

```
# — Dataverse —  
# API key (lasciare vuoto per dataset pubblici)  
DATAVERSE_API_KEY=  
  
# Cartella di destinazione (Transfer Source Location di Archivematica)  
# Recuperabile con: python3 archivematica_ingest.py --list-sources  
OUTPUT_DIR=/mnt/e/DROPBBOX/Dropbox/_ARKIVE/TRANSFER_SOURCE  
  
# Esportazione DCAT-AP (dcat.json) e Schema.org (schema.json)  
# per ogni versione del dataset. Equivalente al flag --dcat da CLI.  
EXPORT_DCAT=false
```

```

# — Archivematica Dashboard —————
AM_URL=https://192.168.139.39
AM_USER=zfed
AM_API_KEY=

# — Archivematica Storage Service —————
SS_URL=https://192.168.139.39:8000
SS_USER=archivematica
SS_API_KEY=

# — Transfer —————
AM_TRANSFER_SOURCE_UUID=
AM_TRANSFER_TYPE=standard
AM_PROCESSING_CONFIG=dataverse_001

# Copia locale VERSIONATA del processing config XML (RACCOMANDATO).
# Elimina la dipendenza dalla shared directory di Archivematica
# (/var/archivematica/...), i cui permessi possono cambiare con
# update/riavvii. Setup: vedi sezione 5.
AM_PROCESSING_MCP_PATH=/home/zfed/arkive/config/dataverse_001ProcessingMCP.xml

AM_AUTO_APPROVE=true

# Timeout API in secondi (default: 120). Con dataset grandi o
# Archivematica sotto carico servono valori generosi; gli errori
# di rete transitori vengono comunque ritentati.
AM_API_TIMEOUT=120

# — SSL —————
SSL_VERIFY=false

```

Recuperare i valori mancanti

```

# UUID Transfer Source
python3 archivematica_ingest.py --no-verify-ssl --list-sources

# Processing configuration disponibili
python3 archivematica_ingest.py --no-verify-ssl --list-processing-configs

```

Priorita' delle variabili

Priorita'	Fonte
1 (massima)	Argomento da riga di comando
2	Variabile d'ambiente di sistema (export)

Priorita'	Fonte
3	File <code>.env</code>
4	Default nel codice

Attenzione: se in una sessione precedente hai eseguito `export $(grep -v '^#' .env | xargs)`, le variabili rimangono attive e hanno priorit  sul `.env`. In caso di comportamenti inattesi, aprire una nuova sessione shell.

Sicurezza

```
echo ".env" >> .gitignore
```

Errori comuni nel `.env`

Errore	Sintomo	Correzione
<code>SS_URL=https://...</code>	MissingSchema	<code>SS_URL=https://...</code>
<code>SS_URL=http://...</code>	302 Found	<code>SS_URL=https://...</code>
Chiavi API vuote	401 Unauthorized	Compilare le chiavi
UUID errato	404 Not Found	Usare <code>--list-sources</code>
Variabili esportate in sessione precedente	Valori vecchi	Nuova sessione shell

4. Script 1 — `scarica_dataverse.py`

Cosa fa

Per ogni DOI elencato in `dois.txt`:

- Recupera l'elenco di tutte le versioni pubblicate
- Controlla se il dataset contiene file:** se   vuoto, non crea la directory e registra il DOI in `doi_vuoti.json`
- Per ciascuna versione:
 - scarica tutti i file del dataset — per i file **ingeriti come tabelari** scarica l'**originale** depositato (`format=original`), non la derivata `.tab` — **verificando l'integrit ** di ognuno contro il checksum (o la dimensione) dichiarato da Dataverse
 - salva i metadati Dataverse in `dataverse.json`
 - genera `metadata.csv` Dublin Core per quella versione
 - genera `dcat.json` (DCAT-AP JSON-LD) se `EXPORT_DCAT=true`
 - scarica `schema.json` (Schema.org JSON-LD) se `EXPORT_DCAT=true`
- Genera un `metadata.csv` complessivo nella radice del pacchetto (letto da Archivematica per il METS `dmdSec`)

Controllo dataset vuoti

Prima di creare qualsiasi directory, lo script verifica che il dataset contenga almeno un file. Se tutte le versioni sono prive di file:

- la directory del DOI **non viene creata**
- il DOI viene registrato in `doi_vuoti.json` con data e motivo
- il riepilogo mostra **Vuoti: N** nel conteggio finale

Cause tipiche di dataset vuoto: - dataset ancora in bozza (non ancora pubblicato) - accesso ristretto (embargo attivo) - dataset pubblicato ma senza file allegati

Conservazione dell'originale (file tabellari)

Quando un file viene caricato su Dataverse in un formato tabellare riconosciuto (Stata `.dta`, SPSS `.sav`, CSV, `xlsx`...), Dataverse lo **ingerisce**: conserva il file originale depositato e ne genera una rappresentazione derivata `.tab` (tab-delimited), che è quella servita di default dall'API di accesso.

Ai fini della conservazione questa distinzione è decisiva. L'oggetto autentico da preservare secondo OAIS è il **bitstream originale depositato dal ricercatore**, non la normalizzazione prodotta da Dataverse: è Archivematica, in fase di ingest, a generare le derivate di preservazione e accesso secondo il proprio FPR. Inoltre la derivata `.tab` viene **rigenerata a ogni richiesta** e ha un checksum instabile, mentre l'originale è statico e verificabile.

Per questo, quando `_is_ingested_tabular` riconosce un file ingerito (presenza dei campi `originalFileName/originalFileFormat`), la pipeline:

1. scarica l'originale con `.../api/access/datafile/{id}?format=original`;
2. lo salva con il **nome originale** (`02500.csv`, `dati.dta`...), non `.tab` — fondamentale perché Archivematica identifichi correttamente il formato (Siegfried/FITS) e pianifichi la conservazione giusta;
3. ne verifica l'integrità con il checksum del manifest.

Fallback: se `format=original` non è disponibile (alcune versioni Dataverse rispondono `404`), la pipeline ripiega sulla derivata `.tab`, in modalità degradata tracciata a log con `[FALLBACK]`, invece di far fallire il DOI. Sulla `.tab` la verifica del checksum non è applicabile: si controlla solo che il file non sia vuoto.

I file **non** ingeriti (PDF, ZIP, immagini, e ogni file non tabellare) non hanno un originale distinto: vengono scaricati e verificati come sempre, senza `format=original`.

Verifica di integrità dei file

Ogni file scaricato viene confrontato con i metadati di integrità che Dataverse espone nel `dataFile`. Lo scopo è impedire che un download **troncato** (interruzione, timeout, lock transitori su `drvfs/Dropbox`) lasci sul disco un file parziale

che ai run successivi verrebbe considerato “già presente” e non più ri-scariato. Un file monco entrerebbe nel SIP, Archivematica ne calcolerebbe un checksum PREMIS valido su un contenuto errato, e finirebbe nell’AIP: una corruzione silenziosa, il difetto peggiore in un contesto di conservazione OAIS/PREMIS.

Per i file ingeriti il checksum del manifest si riferisce all’**originale** (Dataverse lo calcola al momento del caricamento, prima dell’ingest — verificato empiricamente su 30/30 file campionati di JMKSAW su dataverse.unimi.it). Poiché la pipeline scarica proprio l’originale, la verifica è **fixity forte**. È anche il motivo per cui in passato i `.tab` producevano falsi mismatch di massa: si confrontavano i byte della derivata con il checksum dell’originale.

Il controllo usa, in ordine di preferenza:

1. **Dimensione attesa** come controllo di skip primario (per gli ingeriti `originalFileSize`, altrimenti `fileSize`). Un download troncato si manifesta sempre come byte mancanti, quindi la sola dimensione lo intercetta con un costo minimo (una `stat()`), senza rileggere l’intero inventario a ogni esecuzione — aspetto rilevante su drvfs.
2. **Checksum** come fallback di skip quando la dimensione non è disponibile, e soprattutto come **verifica post-download** su ogni file appena scaricato. Il checksum Dataverse viene letto in entrambi i formati esistenti:
 - formato nuovo: `checksum: {"type": "MD5" | "SHA-1" | "SHA-256", "value": "..."}`
 - formato vecchio: `md5: "..."`con il `type` mappato all’algoritmo `hashlib` corretto (non si assume MD5: al primo run si verifica dal log, cercando se compare `(MD5 ok)` o `(SHA1 ok)` nei `[SKIP]`).
3. **Comportamento storico** (skip se il file esiste e non è vuoto) come ultima spiaggia, solo quando né dimensione né checksum sono disponibili.

Conseguenze pratiche sui file già presenti:

- file **integro** → `[SKIP]` Già' presente (dimensione ok) (o `(MD5 ok)` / `(SHA1 ok)`)
- file **incompleto o corrotto** → `[RISCARICO]` ... e il file viene ri-scariato sovrascrivendo quello difettoso

Se un file appena scaricato **non** supera la verifica post-download, viene **rimosso** e il download è segnalato come errore: la versione risulta **partial** nel `riepilogo_download.json` e nessun file difettoso resta sul disco a essere saltato al run seguente.

Esportazione DCAT-AP e Schema.org

Quando `EXPORT_DCAT=true` (nel `.env`) o `--dcat` (da CLI), per ogni versione vengono generati due file aggiuntivi:

dcat.json — DCAT-AP 2.x JSON-LD generato **localmente** dai metadati già'

scaricati, senza chiamate HTTP aggiuntive. Mapping principale:

Campo DCAT-AP	Sorgente Dataverse
dct:title	title
dct:creator	author (nome + affiliazione)
dcat:contactPoint	datasetContact (vCard)
dct:description	dsDescription
dcat:theme	subject
dcat:keyword	keyword
dct:issued	productionDate / distributionDate
dct:publisher	publisher
dct:license	license / termsOfUse
dcat:distribution	file (formato, dimensione, checksum) — per i file ingeriti descrive l' originale
owl:versionInfo	numero versione (es. 1.0)

Per i file ingeriti come tabellari la `dcat:distribution` descrive coerentemente l'oggetto conservato, cioè l'**originale**: `dct:title` = nome originale, `dcat:mediaType` = `originalFileFormat`, `dcat:byteSize` = `originalFileSize`, `dcat:accessURL` con `?format=original`, e checksum con l'algoritmo dichiarato nel manifest (non più forzato a MD5). Per i file non ingeriti la `distribution` descrive il file così com'è.

schema.json — Schema.org JSON-LD scaricato dall'API Dataverse tramite l'exporter `schema.org`. È la vista *di Dataverse* sul dataset e quindi riflette la rappresentazione di accesso (`.tab`), non necessariamente l'originale che la pipeline conserva: va inteso come copia fedele di ciò che il repository pubblica, non come descrizione dell'AIP.

Opzioni da riga di comando

Opzione	Default	Descrizione
<code>--dois <file></code>	<code>dois.txt</code>	File con l'elenco dei DOI
<code>--output <cartella></code>	dal <code>.env</code> (<code>OUTPUT_DIR</code>)	Cartella di destinazione
<code>--apikey <chiave></code>	dal <code>.env</code>	API key Dataverse
<code>--no-verify-ssl</code>	<code>false</code>	Disabilita verifica SSL
<code>--dcat</code>	<code>false</code>	Abilita esportazione DCAT-AP e Schema.org

Esempi

```
# Standard (tutto da .env)
python3 scarica_dataverse.py

# Con DCAT-AP e Schema.org
python3 scarica_dataverse.py --dcat

# Override cartella di output
python3 scarica_dataverse.py \
  --output /mnt/e/DROPBOX/Dropbox/_ARKIVE/TRANSFER_SOURCE

# Con certificato self-signed
python3 scarica_dataverse.py --no-verify-ssl
```

Output a schermo

```
=====
Dataverse UNIMI -- Download automatico (tutte le versioni)
Sorgente DOI : dois.txt (3 DOI)
Destinazione : /mnt/.../TRANSFER_SOURCE
API key      : *****5678
SSL verify   : False
Export DCAT  : True
=====

[1/3] DOI: doi:10.13130/RD_UNIMI/KS2PUX
--> Recupero lista versioni...
--> 1 versione/i trovata/e: v1.0
-- Versione v1.0 [RELEASED]
[metadata] dataverse.json salvato
[metadata] dcat.json generato
[metadata] schema.json salvato
[data] 4 file trovati, inizio download...
  Scarico: works.tab ... OK (25 KB)
[metadata] metadata.csv (versione) generato (1 righe)
[metadata] metadata.csv generato (1 righe, 1 versione/i)
[validazione] metadata.csv OK
OK Stato: ok | Versioni: 1 | File OK: 4 | Errori: 0

[2/3] DOI: doi:10.13130/RD_UNIMI/VU0T0
--> Recupero lista versioni...
--> 1 versione/i trovata/e: v1.0
[INFO] Dataset vuoto: 1 versione/i trovata/e ma nessun file presente
[INFO] Directory non creata.
0 Stato: empty | Versioni: 0 | File OK: 0 | Errori: 0
```

Motivo: 1 versione/i trovata/e ma nessun file presente

```
=====
RIEPILOGO FINALE
Completati   : 2
Parziali     : 0
Errori       : 0
Vuoti        : 1
Riepilogo    : .../riepilogo_download.json
DOI vuoti    : .../doi_vuoti.json
=====
```

Comportamento idempotente

Lo script può essere rieseguito senza duplicare il lavoro già fatto:

- File già scaricati: **saltati solo se superano la verifica di integrità** (vedi “Verifica di integrità dei file”). Un file integro produce [SKIP] Già' presente (dimensione ok); un file incompleto o corrotto viene invece [RISCARICO] e ri-scaricato. Questo rende la ri-esecuzione anche un modo per **sanare** una cartella rimasta con download parziali dopo un'interruzione. Nota: i file ingeriti sono riconosciuti per **nome originale** (02500.csv), quindi cartelle popolate da run precedenti a questa modifica — che contengono i vecchi .tab — non vengono riconosciute (vedi troubleshooting)
- dataverse.json, dcat.json, schema.json, metadata.csv di versione: generati solo se mancanti
- metadata.csv complessivo: **sempre rigenerato**
- doi_vuoti.json: **aggiornato incrementalmente** (le voci precedenti vengono preservate, non sovrascritte)

5. Script 2 — archivematica_ingest.py

Cosa fa

All'avvio, PRIMA di elaborare qualunque pacchetto, lo script verifica (**fail-fast**) che il file XML della processing configuration sia leggibile. Se non lo è, si interrompe con exit code 1: senza `processingMCP.xml` i pacchetti verrebbero processati con la configurazione di **default** di Archivematica (es. con scansione virus attiva), e il problema emergerebbe solo a ingest avviato — molto più costoso da diagnosticare.

Poi, per ogni pacchetto DOI nella Transfer Source Location non ancora completato:

1. Copia `processingMCP.xml` nella radice del pacchetto e **verifica** che la copia

sia avvenuta (in caso contrario il pacchetto viene marcato **failed** e il transfer NON parte)

2. Avvia il **Transfer** con nome AIP-`<YYYYMMDD>-<DOI>`
3. Approva automaticamente il transfer tramite API (`/api/transfer/unapproved/`
 - `/api/transfer/approve/`)
4. Attende il completamento del Transfer (polling)
5. Attende il completamento dell'**Ingest** (SIP -> AIP)
6. Salva l'UUID dell'AIP nel file di stato

Ogni cartella DOI viene trasferita come **un unico pacchetto** con tutte le sue versioni.

Gestione di timeout ed errori di rete transitori

Un timeout di lettura NON significa che l'operazione sia fallita lato Archivematica — significa solo che la risposta non e' arrivata in tempo. Per questo:

- Le chiamate di **polling** e le altre GET vengono ritentate automaticamente (3 tentativi, 15s di pausa) in caso di **Read timed out** o errore di connessione.
- Se **start_transfer va in timeout**, la richiesta potrebbe comunque essere arrivata: con dataset grandi la copia nella watched directory puo' superare il timeout. Lo script cerca allora il transfer per nome tra quelli in attesa di approvazione e, se lo trova, si **RIAGGANCI**A e prosegue normalmente. Solo se il transfer non esiste davvero il pacchetto viene marcato **failed**. Questo evita transfer zombie in Dashboard e AIP duplicati al retry.
- Il timeout e' configurabile con `AM_API_TIMEOUT` nel `.env` o `--api-timeout` da CLI (default: 120s).

Se nonostante tutto un transfer viene completato manualmente dalla Dashboard dopo che lo script lo ha dichiarato **failed**, usare `riconcilia_stato.py` (vedi sezione 7) per allineare il file di stato PRIMA del lancio successivo, altrimenti il pacchetto verrebbe riprocessato da zero creando un AIP duplicato.

Sorgente del processing config: copia locale versionata (RACCOMANDATO)

Il file XML puo' essere letto da due sorgenti, in ordine di priorita':

1. **Copia locale versionata** — `--processing-mcp-path` o `AM_PROCESSING_MCP_PATH` nel `.env`. Nessuna dipendenza dalla shared directory di Archivematica, i cui permessi possono cambiare con update, riavvii dei servizi o salvataggi della configurazione dal Dashboard. La copia puo' essere committata nel repository ed e' riutilizzabile su altri server della pipeline.
2. **Fallback legacy** — `<PROCESSING_CONFIGS_DIR>/<nome>ProcessingMCP.xml` nella shared directory di Archivematica, usando il nome passato con `--processing-config`.

Setup una tantum della copia locale:

```
mkdir -p ~/arkive/config
sudo cp /var/archivematica/sharedDirectory/sharedMicroServiceTasksConfigs/processingMCPConfigs/dataverse
sudo chown $(whoami): ~/arkive/config/dataverse_001ProcessingMCP.xml
```

e nel `.env`:

```
AM_PROCESSING_MCP_PATH=/home/<utente>/arkive/config/dataverse_001ProcessingMCP.xml
```

Attenzione: se la processing configuration viene modificata nel Dashboard di Archivematica, la copia locale va riesportata con il comando `sudo cp` qui sopra. E' un evento raro e consapevole, ma va ricordato. Inoltre, dopo aver modificato il `.env`, riesportare l'ambiente nelle sessioni shell attive: `export $(grep -v '^#' .env | xargs)`

Opzioni da riga di comando

Sorgente e stato

Opzione	Default	Descrizione
<code>--source <cartella></code>	da Transfer Source	Directory con i pacchetti
<code>--state-file <file></code>	<code>stato_archivematica.json</code>	File di tracciamento

Dashboard Archivematica

Opzione	Default	Descrizione
<code>--am-url <url></code>	dal <code>.env</code>	URL Dashboard
<code>--am-user <utente></code>	dal <code>.env</code>	Utente Dashboard
<code>--am-apikey <chiave></code>	dal <code>.env</code>	API key Dashboard

Storage Service

Opzione	Default	Descrizione
<code>--ss-url <url></code>	dal <code>.env</code>	URL Storage Service
<code>--ss-user <utente></code>	dal <code>.env</code>	Utente Storage Service
<code>--ss-apikey <chiave></code>	dal <code>.env</code>	API key Storage Service

Transfer

Opzione	Default	Descrizione
<code>--transfer-source <UUID></code>	dal <code>.env</code>	UUID Transfer Source Location

Opzione	Default	Descrizione
--transfer-type <tipo>	standard	Tipo di transfer
--processing-config <nome>	dataverse_001	Processing configuration
--processing-mcp-path <file>	dal .env	Copia locale versionata del processing config XML (RACCOMANDATO); ha priorit� sulla shared directory
--ignore-missing-processing-config	—	Prosegue anche senza processing config XML leggibile (SCONSIGLIATO: i transfer usano il default di Archivemtica)
--auto-approve	dal .env	Approva automaticamente il transfer

Utility

Opzione	Descrizione
--list-sources	Elenca Transfer Source Locations ed esce
--list-processing-configs	Elenca processing configuration ed esce
--no-verify-ssl	Disabilita verifica SSL
--poll-interval <sec>	Intervallo polling (default: 15s)
--api-timeout <sec>	Timeout chiamate API (default: 120s, o AM_API_TIMEOUT nel .env)
--dry-run	Mostra cosa farebbe senza eseguire

Esempi

Standard (tutto da .env)

```
python3 archivematica_ingest.py
```

Lista Transfer Source disponibili

```
python3 archivematica_ingest.py --no-verify-ssl --list-sources
```

Anteprima senza eseguire

```
python3 archivematica_ingest.py --dry-run
```

Processing configuration diversa

```
python3 archivematica_ingest.py --processing-config automated
```

Copia locale del processing config esplicita (senza .env)

```
python3 archivematica_ingest.py --processing-mcp-path ~/arkive/config/dataverse_001ProcessingMCP.xml
```

Output a schermo

```
=====
Archivematica Ingest -- trasferimento automatico dataset
Sorgente      : /mnt/.../TRANSFER_SOURCE
Pacchetti DOI : 3
Dashboard     : https://192.168.139.39 (utente: zfed)
Storage Svc   : https://192.168.139.39:8000 (utente: archivematica)
Transfer Src   : 76ab3067-...
Transfer type  : standard
Processing cfg : dataverse_001
MCP XML       : /home/zfed/arkive/config/dataverse_001ProcessingMCP.xml
Auto-approve  : True
SSL verify    : False
File di stato : stato_archivematica.json
=====
```

```
[1/3] doi_10.13130_RD_UNIMI_KS2PUX
--> Elaborazione: doi_10.13130_RD_UNIMI_KS2PUX
[debug] processingMCP.xml copiato e verificato in .../processingMCP.xml
Avvio transfer 'AIP-20260619-doi_10.13130_RD_UNIMI_KS2PUX' ...
Transfer UUID: <uuid>
[approve] Attendo 10s ... (tentativo 1/5)
[approve] Trovato: AIP-20260619-...-<uuid> -- invio approvazione ...
[approve] Approvato (UUID: <uuid>)
Attendo completamento transfer ...
[transfer] stato: PROCESSING -- attendo 15s ...
SIP UUID: <uuid>
Attendo completamento ingest ...
OK Ingest completato -- AIP UUID: <uuid>
```

```
=====
RIEPILOGO FINALE
Completati   : 3
Gia' presenti: 0
Errori       : 0
=====
```

6. Struttura del pacchetto generato

```
TRANSFER_SOURCE/
  doi_10.13130_RD_UNIMI_KS2PUX/
    objects/
      v1.0/
        objects/                <- file del dataset
        works.tab
        README.txt
        metadata/               <- metadati di questa versione
          dataverse.json        <- metadati Dataverse (preservation)
          metadata.csv          <- Dublin Core di questa versione
          dcat.json             <- DCAT-AP JSON-LD (solo se EXPORT_DCAT=true)
          schema.json           <- Schema.org JSON-LD (solo se EXPORT_DCAT=true)
      LD (solo se EXPORT_DCAT=true)
      v2.0/
        objects/
          works_v2.tab
        metadata/
          dataverse.json
          metadata.csv
          dcat.json             <- (solo se EXPORT_DCAT=true)
          schema.json          <- (solo se EXPORT_DCAT=true)
        metadata/
          metadata.csv          <- Dublin Core TUTTE le versioni -
      > METS dmdSec
        processingMCP.xml       <- copiato da archivematica_ingest.py
        riepilogo_download.json <- riepilogo scaricamenti
        doi_vuoti.json          <- DOI senza file (aggiornato incrementalmente)
```

Formato metadata.csv (radice del pacchetto)

filename	dc.title	dc.creator	dc.date	dc.identifier	...
objects/v1.0/objects/works_v1.tab	File di lavoro	Autore	2026-03-04	doi:10.13130/...	
objects/v2.0/objects/works_v2.tab	File di lavoro	Autore	2026-03-04	doi:10.13130/...	

Archivematica traspone questi metadati nel METS.xml come `<dmdSec MD-TYPE="DC">`.

7. File di stato e report

riepilogo_download.json

Generato da `scarica_dataverse.py`. Riepilogo completo di ogni DOI:


```
[
  {
    "doi": "doi:10.13130/RD_UNIMI/KS2PUX",
    "status": "ok",
    "versions": [{"version": "v1.0", "files_ok": 4, "files_err": 0}],
    "error": null,
    "empty_reason": null
  },
  {
    "doi": "doi:10.13130/RD_UNIMI/VUOTO",
    "status": "empty",
    "versions": [],
    "error": null,
    "empty_reason": "1 versione/i trovata/e ma nessun file presente"
  }
]
```

doi_vuoti.json

Generato da `scarica_dataverse.py`. Contiene solo i DOI senza file, aggiornato incrementalmente ad ogni esecuzione:

```
[
  {
    "doi": "doi:10.13130/RD_UNIMI/VUOTO",
    "empty_reason": "1 versione/i trovata/e ma nessun file presente (dataset vuoto o accesso ristretto)",
    "rilevato_il": "2026-06-19T10:30:00+00:00"
  }
]
```

stato_archivematica.json

Generato da `archivematica_ingest.py`. Traccia lo stato di ogni transfer:

```
{
  "doi_10.13130_RD_UNIMI_KS2PUX": {
    "transfer_name": "AIP-20260619-doi_10.13130_RD_UNIMI_KS2PUX",
    "transfer_uuid": "bde477ff-...",
    "sip_uuid": "a1b2c3d4-...",
    "aip_uuid": "e5f6a7b8-...",
    "status": "ingested",
    "completed_at": "2026-06-19T10:42:13+00:00",
    "error": null
  }
}
```

Valori di status (stato_archivematica.json)

Valore	Significato
<code>in_progress</code>	Avviato ma non completato
<code>transferred</code>	Transfer ok, ingest non completato
<code>ingested</code>	Completato – saltato nelle esecuzioni successive
<code>failed</code>	Errore – ritentato da zero al prossimo avvio

Riconciliazione dello stato — `riconcilia_stato.py`

Quando un pacchetto viene completato con un **intervento manuale** in Dashboard (es. approvazione a mano dopo un timeout dello script), Archivematica lo porta correttamente a AIP ma il file di stato lo registra ancora come `failed`, senza AIP UUID. Un lancio successivo di `archivematica_ingest.py` lo riprocesserebbe da zero, creando un **AIP duplicato** per lo stesso DOI.

`riconcilia_stato.py` risolve: per ogni pacchetto `failed` interroga lo Storage Service, cerca un AIP con status `UPLOADED` il cui `current_path` contenga il DOI, e — solo se il match e' **univoco** — aggiorna la voce a `ingested` con l'AIP UUID reale. Con 0 o 2+ match la voce non viene toccata e viene segnalata per verifica manuale.

Anteprima senza modifiche (consigliata come primo passo)

```
python3 riconcilia_stato.py --dry-run
```

Applica (crea automaticamente un backup del file di stato)

```
python3 riconcilia_stato.py
```

Solo DOI specifici (frammenti, separati da virgola)

```
python3 riconcilia_stato.py --dois H4W0JR,NZRXC
```

Configurazione: legge `SS_URL`, `SS_USER`, `SS_API_KEY`, `SSL_VERIFY` dal `.env` (stessa priorita' della pipeline: `CLI > export > .env`). Prima di ogni scrittura crea `stato_archivematica.json.bak_<timestamp>`. Exit code: 0 tutto riconciliato, 2 se restano voci ambigue.

Reset manuale di un pacchetto

```
python3 -c "
import json
s = json.load(open('stato_archivematica.json'))
s.pop('doi_10.13130_RD_UNIMI_KS2PUX', None)
json.dump(s, open('stato_archivematica.json', 'w'), indent=2)
"
```

Reset completo

```
echo '{}' > stato_archivematica.json
```

8. Risoluzione dei problemi comuni

MissingSchema / URL non valido

MissingSchema: Invalid URL 'https://...'

Aggiungere i due punti: https:// non https/.

302 Found sulla porta 8000

`SS_URL=https://192.168.139.39:8000`

401 Unauthorized

Verificare `AM_API_KEY` e `SS_API_KEY` nel `.env`.

404 Not Found sulla Transfer Source UUID

`python3 archivematica_ingest.py --no-verify-ssl --list-sources`

Aggiornare `AM_TRANSFER_SOURCE_UUID` nel `.env` con l'UUID corretto per il server in uso (ogni installazione Archivematica ha UUID diversi).

SSL Certificate Verify Failed

`SSL_VERIFY=false` # nel `.env`

oppure

`python3 archivematica_ingest.py --no-verify-ssl`

`EXPORT_DCAT=true` nel `.env` ma `dcat.json` non viene generato

Verificare che non ci siano variabili esportate in precedenza con `export` che sovrascrivono il `.env`:

`unset EXPORT_DCAT`

`python3 scarica_dataverse.py --dcat` # oppure rilancia in una nuova sessione

[RISCARICO] ripetuti o “checksum non combacia” su un file

Durante il download `scarica_dataverse.py` confronta ogni file con il checksum (o la dimensione) dichiarato da Dataverse. Alcuni casi:

- [RISCARICO] Dimensione non combacia su un file **già presente**: la copia locale è più piccola dell'attesa, tipicamente un download interrotto in precedenza. È il comportamento voluto: il file viene ri-scariato. Nessuna azione necessaria.

- **ERRORE (... non combacia)** su un file **appena scaricato**, che si ripete a ogni run: il download arriva sistematicamente corrotto o troncato. Verificare la stabilità della rete/proxy e, se il file è grande, che non si stia colpendo un limite lato server. Il file difettoso viene rimosso a ogni tentativo, quindi non entra mai nel pacchetto.
- Nel log compare **[SKIP] Già' presente (nessuna verifica disponibile)**: Dataverse non ha esposto né dimensione né checksum per quel file. Lo skip ricade sul comportamento storico (esiste e non è vuoto); in questo caso l'integrità non è garantita dallo script e va eventualmente verificata a valle.

Per sapere quale algoritmo di checksum usa davvero l'istanza, osservare la dicitura tra parentesi nei messaggi di **[SKIP]** (**(MD5 ok)**, **(SHA1 ok)**, ...).

[FALLBACK] salvata derivata .tab

Compare quando, per un file ingerito come tabellare, **format=original** non è disponibile (tipicamente un **404** su alcune versioni Dataverse). La pipeline ripiega sulla derivata **.tab** invece di far fallire il DOI. È una modalità **degradata**: il pacchetto conterrà la **.tab** (non verificabile via checksum) invece dell'originale. Se il messaggio è isolato, è tollerabile; se è sistematico su un'istanza, conviene verificare con l'amministratore Dataverse perché **format=original** non risponde, dato che l'obiettivo di conservazione è l'originale.

Ho ri-scaricato e mi ritrovo sia i .csv/.dta sia i vecchi .tab

Da quando la pipeline conserva gli originali, i file ingeriti vengono salvati col nome originale (**02500.csv**), non più **02500.tab**. Su una cartella di download popolata da un run **precedente** a questa modifica, un nuovo run non riconosce i **.tab** come “già presenti” (cerca **02500.csv**) e scarica gli originali **accanto** ai vecchi **.tab**. Non è un errore del codice, è la conseguenza del cambio di nome. Prima di rilanciare su cartelle già popolate in passato, svuotarle (o rimuovere i **.tab** residui). La migrazione dei pacchetti/AIP già prodotti con i **.tab** è un tema a sé, da gestire con una procedura dedicata.

[ERRORE] Impossibile leggere il processing config

Dalla versione corrente lo script si interrompe (**fail-fast**) se il file XML della processing configuration non è leggibile, invece di proseguire e produrre pacchetti processati con la configurazione di default di Archivematica (es. con scansione virus attiva).

Soluzione raccomandata — copia locale versionata, indipendente dai permessi della shared directory di Archivematica:

```
mkdir -p ~/arkive/config
sudo cp /var/archivematica/sharedDirectory/sharedMicroServiceTasksConfigs/processingMCPConfigs/dataverse.xml ~/arkive/config/
sudo chown $(whoami): ~/arkive/config/dataverse_001ProcessingMCP.xml
```

```
# nel .env:
# AM_PROCESSING_MCP_PATH=/home/<utente>/arkive/config/dataverse_001ProcessingMCP.xml
```

Alternative (fragili: i permessi possono ri-cambiare a ogni update/riavvio di Archivematica):

```
sudo usermod -aG archivematica <utente>
# Poi fare logout e login: la membership nel gruppo ha effetto solo
# sulle NUOVE sessioni. Sessioni tmux/screen di lunga data o job cron
# avviati prima dell'aggiunta NON vedono il gruppo.
```

Per proseguire consapevolmente senza processing config (SCONSIGLIATO): --ignore-missing-processing-config.

[ERRORE] Copia di processingMCP.xml nel pacchetto fallita (Permission denied sulla DESTINAZIONE)

Diverso dal caso precedente: qui la sorgente e' leggibile ma la scrittura nella radice del pacchetto fallisce. Il pacchetto viene marcato **failed** e il transfer NON parte; gli altri pacchetti proseguono normalmente. Tipico con Transfer Source su mount Windows/Dropbox (WSL drvfs). Cause:

1. **processingMCP.xml** gia' presente e non scrivibile nella cartella (es. residuo di una copia manuale con `sudo`, o attributo read-only di Windows). Lo script rimuove automaticamente la destinazione preesistente prima della copia.
2. **Lock transitorio del client Dropbox** durante la sincronizzazione del file. Lo script ritenta automaticamente 3 volte con 2s di pausa.

Se l'errore persiste dopo i retry automatici:

```
# Verifica il file incriminato
ls -l <TRANSFER_SOURCE>/<doi>/processingMCP.xml
```

```
# Elenca i pacchetti falliti nel file di stato
python3 -c "import json; s=json.load(open('stato_archivematica.json')); \
print([k for k,v in s.items() if v.get('status')=='failed'])"
```

Poi rilancia `archivematica_ingest.py`: i pacchetti **failed** vengono ritentati automaticamente da zero.

Buona pratica: durante esecuzioni massive, mettere in PAUSA la sincronizzazione Dropbox (o escludere `TRANSFER_SOURCE` dal sync) riduce drasticamente i lock transitori e le collisioni.

Nota: un "Permission denied" puo' presentarsi anche con permessi apparentemente corretti su file e directory: verificare il permesso di attraversamento (x) su TUTTE le directory del percorso, eventuali ACL (`getfacl`), filesystem di rete (NFS/CIFS con squash degli

UID), e che la sessione da cui gira lo script abbia effettivamente la membership nei gruppi attesa (id nella STESSA sessione).

Transfer REJECTED al riavvio

Lo script rileva automaticamente status `failed` e riparte da zero.

Read timed out durante transfer/ingest

Con dataset molto grandi o con Archivematica sotto carico (molti ingest in coda), le risposte API possono superare il timeout. Lo script gestisce automaticamente il caso (retry sulle GET, riaggancio del transfer dopo timeout su `start_transfer` — vedi sezione 5). Se i timeout sono ricorrenti, aumentare `AM_API_TIMEOUT` nel `.env` (o `--api-timeout`).

Se un pacchetto viene comunque marcato `failed` ma in Dashboard il transfer prosegue e arriva in fondo (anche con approvazione manuale), usare `riconcilia_stato.py` per allineare il file di stato PRIMA del lancio successivo (vedi sezione 7).

Transfer “zombie” in Dashboard (in attesa, non rimovibili)

Transfer rimasti in attesa di approvazione che la Dashboard non riesce a rifiutare o rimuovere (tipicamente dopo timeout o riavvii di MCPServer). Sintomi: le azioni nel menu non hanno effetto, oppure il bollino con il conteggio delle decisioni pendenti resta acceso anche dopo aver nascosto le voci.

Lezione chiave: il flag `hidden` nel database nasconde la voce dalla UI ma NON ferma la lavorazione. Finché il pacchetto resta fisicamente nelle watched directories, **MCPServer ricrea i job di approvazione a ogni riavvio** (riconoscibili: `createdTime` coincidente con l’orario del riavvio). La rimozione va fatta nell’ordine giusto:

```
# 1. Identifica UUID e percorso fisico dei transfer da rimuovere
mysql -u archivematica -p MCP -e "
SELECT transferUUID, hidden, currentLocation FROM Transfers
WHERE currentLocation REGEXP '<frammento1>|<frammento2>';"

# 2. FERMA MCPServer (altrimenti ricrea i job mentre pulisci)
sudo systemctl stop archivematica-mcp-server

# 3. Rimuovi le cartelle indicate da currentLocation
# (%sharedPath% = /var/archivematica/sharedDirectory/)
# ATTENZIONE: solo quelle, verificando che i nomi contengano i DOI attesi.
# Gli originali in TRANSFER_SOURCE non vengono toccati (start_transfer
# lavora su una COPIA).

# 4. Backup e pulizia del database
```

```
mysqldump --no-tablespaces -u archivematica -p MCP Transfers Jobs \
> /tmp/backup_transfers_jobs_$(date +%Y%m%d_%H%M).sql
mysql -u archivematica -p MCP -e "
UPDATE Transfers SET hidden=1 WHERE transferUUID IN ('<uuid1>', '<uuid2>');
UPDATE Jobs SET currentStep=4 WHERE currentStep=1
AND SIPUUID IN ('<uuid1>', '<uuid2>');"
# (currentStep: 1 = awaiting decision, 4 = failed)

# 5. Riavvia e verifica
sudo systemctl start archivematica-mcp-server
sudo systemctl restart archivematica-dashboard
mysql -u archivematica -p MCP -e \
"SELECT COUNT(*) FROM Jobs WHERE currentStep = 1;" # atteso: 0, e deve RESTARE 0
```

Nota su mysqldump: senza `--no-tablespaces` l'utente archivematica riceve "Access denied; you need PROCESS privilege" — il flag e' innocuo per il backup di singole tabelle.

Permission denied sulla copia di processingMCP.xml nei pacchetti (cartelle di proprieta' altrui)

Se alcune cartelle nella Transfer Source appartengono a un altro utente (es. archivematica:archivematica con permessi 755), la copia di processingMCP.xml fallisce con Permission denied nonostante retry. Causa tipica: script lanciati in passato con `sudo -u archivematica` o come root. **Mai lanciare gli script della pipeline con un utente diverso da quello operativo.** Rimedio:

```
# Riporta all'utente operativo la proprieta' delle cartelle fallite
# (basta la radice del pacchetto; il gruppo archivematica resta per la lettura)
sudo chown <utente>:archivematica <TRANSFER_SOURCE>/<doi>/
```

400 "Unable to determine the status of the unit"

Errore transitorio: lo script ritenta automaticamente 2 volte (20s di attesa).

Auto-approve non funziona

1. Verificare `AM_AUTO_APPROVE=true` nel `.env`
2. Verificare che il transfer sia in `USER_INPUT` nel Dashboard
3. Aumentare `APPROVE_TRANSFER_WAIT` nel codice (default: 10s) se Archivematica e' lenta ad avviare il transfer

Elasticsearch non raggiungibile

```
sudo systemctl start elasticsearch
curl -s http://localhost:9200
```

9. Esempio di esecuzione completa

Configurazione iniziale (una tantum)

```
git clone https://github.com/zfed/Arkive.git
cd Arkive/tools/dataverse-archivematica

pip install requests --break-system-packages

cp .env.template .env
nano .env
# Compilare almeno:
#   OUTPUT_DIR, DATAVERSE_API_KEY, AM_API_KEY, SS_API_KEY,
#   AM_TRANSFER_SOURCE_UUID, SSL_VERIFY

# Recupera UUID Transfer Source
python3 archivematica_ingest.py --no-verify-ssl --list-sources

# Verifica processing configuration
python3 archivematica_ingest.py --no-verify-ssl --list-processing-configs
```

Esecuzione standard

```
# 1. Prepara la lista DOI
cat > dois.txt << 'EOF'
doi:10.13130/RD_UNIMI/KS2PUX
doi:10.13130/RD_UNIMI/NFURUT
doi:10.13130/RD_UNIMI/QBRJNN
EOF

# 2. Scarica i dataset (OUTPUT_DIR letto dal .env)
python3 scarica_dataverse.py

# 2b. Con metadati DCAT-AP e Schema.org
python3 scarica_dataverse.py --dcat

# 3. Verifica i dataset vuoti (se presenti)
cat doi_vuoti.json | python3 -m json.tool

# 4. Ingesta in Archivematica (completamente automatico)
python3 archivematica_ingest.py

# 5. Verifica lo stato
cat stato_archivematica.json | python3 -m json.tool

# 6. (solo se e' servito un intervento manuale in Dashboard)
#   Riconcilia il file di stato con gli AIP reali
```



```
python3 riconcilia_stato.py --dry-run && python3 riconcilia_stato.py
```

Per le esecuzioni successive, ripetere i passi 2 e 4: gli script elaborano solo cio' che e' nuovo o non ancora completato.